

File di testo in C

Far scrivere ai programmi i propri risultati, per poterli poi disegnare

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Come salvo i risultati?

grave.c stampa 22 righe che scorrono via: come le conservo per usarle dopo?

Terminale: tutto a schermo

```
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.1
   t        y
  0.00   20.00
  0.10   19.95
  0.20   19.80
  0.30   19.56
   ...
  2.00    0.38
  2.10   -1.63
```

Terminale: con la redirezione >

```
$ ./app.exe > grave.txt
20
0.1
$ cat grave.txt
altezza h (m): passo dt (s):    t          y
  0.00   20.00
  0.10   19.95
  0.20   19.80
   ...
  2.00    0.38
  2.10   -1.63
```

Scorre via

22 righe sullo schermo: le ho perse appena le leggo

Con > c'è tutto

anche i prompt e l'intestazione, e non vedo cosa chiede

Meglio così

il programma scrive su un file solo i dati

Scrivere su un file: tre passi

Come con un quaderno: lo si apre, ci si scrive, lo si chiude

1 fopen

apre il file in scrittura ("w") e dà un «segnalibro» da usare dopo

2 fprintf

scrive nel file, tante volte quante servono

3 fclose

chiude il file: i dati sono al sicuro sul disco

i tre passi

```
FILE *fp;
fp = fopen("grave.txt", "w");
fprintf(fp, "%.2f %.2f\n", t, y);
fclose(fp);
```

FILE *fp

la variabile che rappresenta il file aperto: per ora si scrive così

fprintf(fp, ...)

come printf, con il file come primo argomento

Il primo programma che scrive un file

fibonacci_file.c: n, F(n) e F(n+1)/F(n) per n da 1 a 30, tutto in fibonacci.txt

fibonacci_file.c

```c
#include <stdio.h>
#define NMAX 30

int main() {
    FILE *fp;
    int n;
    unsigned long long a0 = 0, a1 = 1, an;  // F(0), F(1)
    fp = fopen("fibonacci.txt", "w");
    for (n = 1; n <= NMAX; n++) {
        an = a0 + a1;                       // F(n+1)
        fprintf(fp, "%d %llu %.6f\n", n, a1, (double) an / a1);
        a0 = a1;
        a1 = an;
    }
    fclose(fp);
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe
fibonacci_file.c
$ ./app.exe
$ head -4 fibonacci.txt
1 1 1.000000
2 1 2.000000
3 2 1.500000
4 3 1.666667
$ tail -2 fibonacci.txt
29 514229 1.618034
30 832040 1.618034
```

%d %llu %.6f

tre colonne separate da spazio, una riga per ogni n

A schermo non esce nulla: i numeri sono tutti nel file, e il rapporto tende a 1.618034 (sezione aurea, M06).

E se il file non si apre?

Qui provo a scrivere in dati/fibonacci.txt, ma la cartella dati non esiste

fibonacci_controllo.c (righe nuove in verde)

```c
FILE *fp;
int n;
unsigned long long a0 = 0, a1 = 1, an;  // F(0), F(1)
fp = fopen("dati/fibonacci.txt", "w"); // non esiste
if (fp == NULL) {
    printf("errore: impossibile aprire il file\n");
    exit(-1);
}
```

Senza controllo (fibonacci_errore.c)

```
$ ./app.exe
Segmentation fault (core dumped)
```

Con controllo

```
$ ./app.exe
errore: impossibile aprire il file
$ echo $?
255
```

NULL

fopen non è riuscita ad aprire il file

exit(-1)

ferma subito il programma (serve stdlib.h)

echo $?

mostra il codice di uscita: -1 diventa 255

Cosa succede se rieseguo?

Con "w" il file viene riscritto da capo, non allungato

Terminale

```
$ ./app.exe
$ ls -l fibonacci.txt
-rw-r--r--@ 1 rahatlou  staff  483 Oct 28 10:54
fibonacci.txt
$ wc -l fibonacci.txt
      30 fibonacci.txt
$ ./app.exe
$ ls -l fibonacci.txt
-rw-r--r--@ 1 rahatlou  staff  483 Oct 28 10:54
fibonacci.txt
$ wc -l fibonacci.txt
      30 fibonacci.txt
```

ls -l file

dimensione in byte e data dell'ultima modifica

cat file

tutto il contenuto del file

head / tail

le prime o le ultime righe

wc -l file

quante righe ha il file

Stessa dimensione, stesse 30 righe: a ogni esecuzione fopen con "w" azzera il file e ricomincia.

La caduta del grave su file

grave_file.c: come grave.c di M05, ma i dati vanno in grave.txt

grave_file.c (righe nuove in verde)

```c
int main() {
    FILE *fp;
    double h, dt, t, y;
    printf("altezza h (m): ");
    scanf("%lf", &h);
    printf("passo dt (s): ");
    scanf("%lf", &dt);
    fp = fopen("grave.txt", "w");
    if (fp == NULL) {
        printf("errore: impossibile aprire grave.txt\n");
        exit(-1);
    }
    t = 0.;
    y = h;
    while (y > 0.) {
        y = h - 0.5 * GRAV * t * t;
        fprintf(fp, "%.2f %.2f\n", t, y);
        t += dt;
    }
    fclose(fp);
}
```

Terminale

```
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.1
$ head -3 grave.txt
0.00 20.00
0.10 19.95
0.20 19.80
$ tail -2 grave.txt
2.00 0.38
2.10 -1.63
$ wc -l grave.txt
      22 grave.txt
```

Ultima riga: y < 0

l'ultimo passo è già sotto terra, come in M05: lo si rivede nel grafico

Che file serve a Python?

Convenzione del corso: colonne di numeri separate da spazio, senza testo

Va bene: grave.txt

```
$ head -4 grave.txt
0.00 20.00
0.10 19.95
0.20 19.80
0.30 19.56
```

No: il file di grave_stampa.c con >

```
$ head -4 grave_male.txt
altezza h (m): passo dt (s):    t          y
  0.00   20.00
  0.10   19.95
  0.20   19.80
```

una riga per ogni punto

colonne separate da uno spazio

solo numeri: niente testo, unità, intestazioni

il punto decimale, non la virgola

una riga con # è un commento: viene saltata

In M09, in Python:

```
t, y = np.loadtxt('grave.txt', unpack=True)
```

legge le due colonne in due array t e y

Esercizi

Da fare a casa: i file prodotti servono agli esercizi della lezione sui grafici

1 Pendolo

Scrivi pendolo.txt con theta(t) = theta0 cos(omega t), per t da 0 a 5 s con passo 0.1. Colonne t e theta (%.2f %.4f).

prova con:

```
theta0 omega
0.3 2.5
0.5 1.2
```

2 Capitale

Leggi capitale e tasso annuo in percento (do/while). Scrivi anno e capitale per 30 anni in capitale.txt (%d %.2f).

prova con:

```
capitale tasso
1500 3.5
800 7
```

3 Cubi

Leggi n tra 1 e 50 (do/while) e scrivi i, i^2, i^3 per i da 1 a n in cubi.txt. Controlla fopen.

prova con:

```
12
50
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (-lm serve per le funzioni di math.h)

Riepilogo

1	**Un file dal C**	fopen, fprintf, fclose: tre passi; con "w" il file è riscritto da capo ogni volta
2	**Controllare**	fopen può fallire e dare NULL: if (fp == NULL) con messaggio e exit(-1)
3	**Per Python**	file di testo con colonne di numeri separate da spazio, senza testo né intestazioni